EXPERIMENT NO.1

Aim:

To implement **Linear Regression** and **Logistic Regression** using a single real-world dataset and evaluate their performance using appropriate metrics.

Theory:

Linear Regression Theory

Linear Regression is a supervised machine learning algorithm used to model the relationship between one dependent variable and one or more independent variables by fitting a linear equation to observed data. It assumes that the change in the dependent variable is directly proportional to the change in the independent variables.
The main objective of Linear Regression is to find the best-fit straight line (regression line) that minimizes the difference between the predicted values and the actual values. This difference is measured using the Mean Squared Error (MSE). Linear Regression is widely used for prediction and forecasting tasks where the target variable is continuous.

Logistic Regression Theory

Logistic Regression is a supervised machine learning algorithm primarily used for binary classification problems. Despite its name, it is a classification algorithm rather than a regression algorithm. Logistic Regression predicts the probability that a given input belongs to a particular class.
It uses the sigmoid (logistic) function to map predicted values to probabilities between 0 and 1. Based on a threshold (usually 0.5), the output is classified into one of the two classes. Logistic Regression is commonly used in medical diagnosis, spam detection, and risk assessment problems.

1. Dataset Source

- **Dataset Name:** Iowa Housing Dataset
- **Source Link:** https://www.kaggle.com/datasets/vikrishnan/boston-house-prices

2. Dataset Description

The Boston Housing dataset contains information related to housing prices in the Boston metropolitan area.

- **Number of features:** 13
- **Target Variable:** MEDV – Median value of owner-occupied homes (in $1000s)
- **Number of instances:** 506

Usage in This Experiment

- **Linear Regression:** Predicts continuous house prices (MEDV)
- **Logistic Regression:** Converts MEDV into binary classes:
  - 1 → High-priced house (MEDV ≥ median)
  - 0 → Low-priced house (MEDV < median)

3. Mathematical Formulation

Linear Regression

Hypothesis Function:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Cost Function (Mean Squared Error):

$$J(\beta) = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

Logistic Regression

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Hypothesis Function:

$$P(y = 1|x) = \sigma(\beta_0 + \beta_1 x_1 + ... + \beta_n x_n)$$

4. Algorithm Limitations

Linear Regression

- Assumes linear relationship
- Sensitive to outliers
- Poor performance on non-linear data

Logistic Regression

- Suitable mainly for binary classification

- Sensitive to outliers
- Assumes independent features

5. Methodology / Workflow

1. Load dataset
2. Preprocess data
3. Feature scaling
4. Train-test split
5. Train Linear Regression model
6. Convert target into binary classes
7. Train Logistic Regression model
8. Evaluate performance
9. Perform hyperparameter tuning

6. Performance Analysis

Linear Regression Metrics

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Logistic Regression Metrics

- Accuracy
- Confusion Matrix
- Precision, Recall, F1-score

7. Implementation

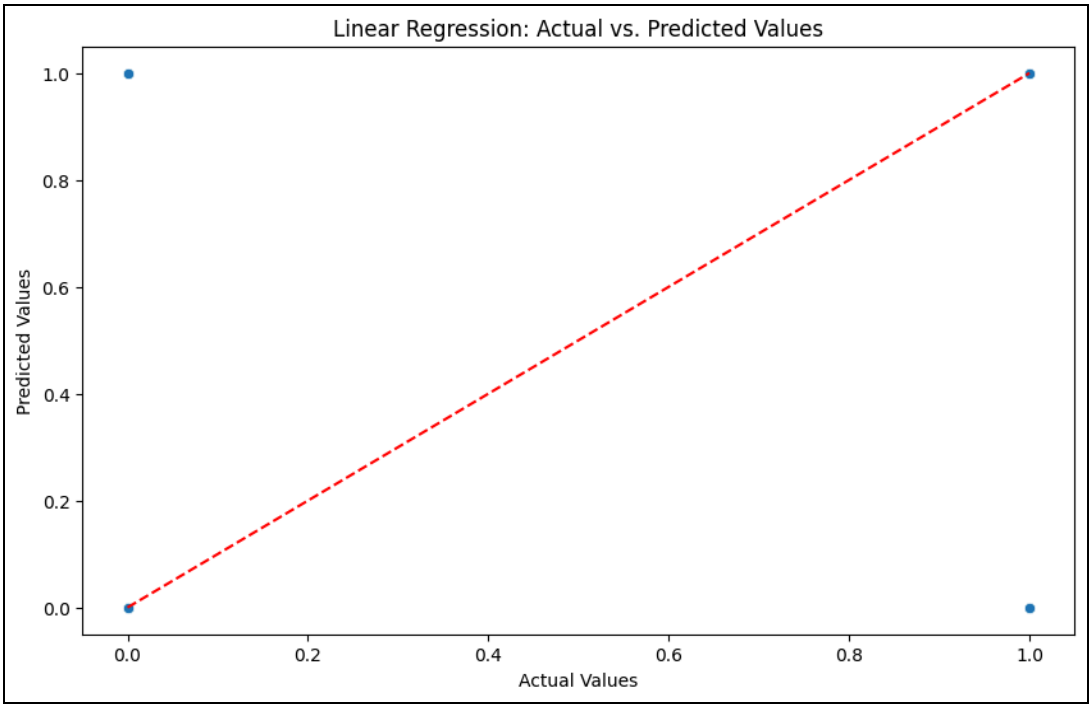
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, confusion_matrix,
classification_report

# Load Ames Housing dataset (local CSV)
data = pd.read_csv("AmesHousing.csv")

# Select numeric features only
```

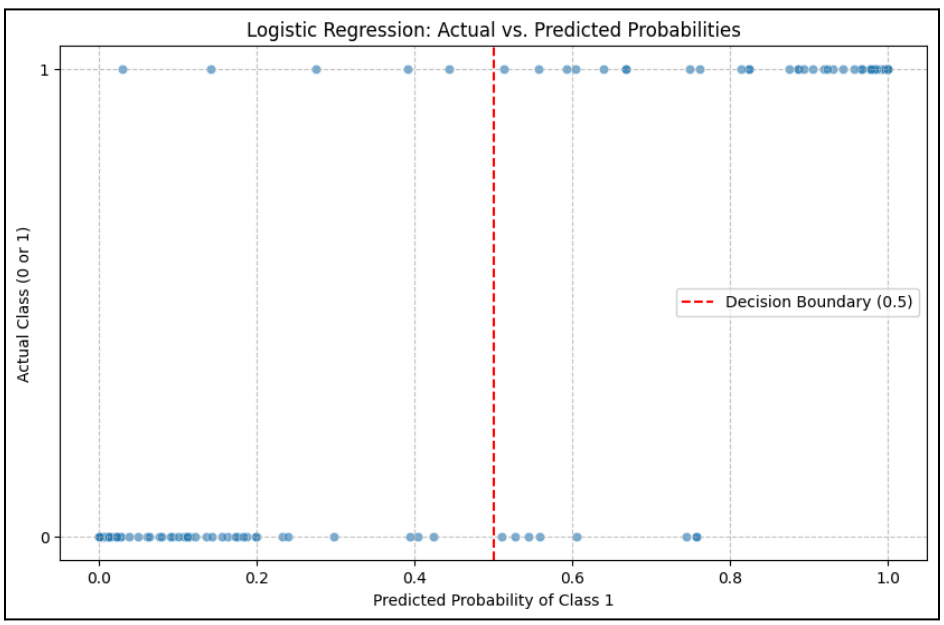
```
data_numeric = data.select_dtypes(include=[np.number])

X = data_numeric.drop("SalePrice", axis=1)
y = data_numeric["SalePrice"]

# ---------------- Linear Regression ----------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
y_pred = lin_model.predict(X_test)

print("Linear Regression Results")
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))

# ---------------- Ridge Regression ----------------
ridge = Ridge()
params_ridge = {'alpha': [0.01, 0.1, 1, 10, 100]}

grid_ridge = GridSearchCV(ridge, params_ridge, cv=5)
grid_ridge.fit(X_train, y_train)

print("Best Ridge Alpha:", grid_ridge.best_params_)

# ---------------- Logistic Regression ----------------
# Convert prices into high vs low price houses
median_price = np.median(y)
y_binary = (y >= median_price).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y_binary, test_size=0.2, random_state=42
)

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
log_model = LogisticRegression(max_iter=500)
log_model.fit(X_train, y_train)
y_pred = log_model.predict(X_test)

print("\nLogistic Regression Results")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

params_log = {'C': [0.01, 0.1, 1, 10]}
grid_log = GridSearchCV(LogisticRegression(max_iter=500), params_log, cv=5)
grid_log.fit(X_train, y_train)

print("Best Logistic Parameters:", grid_log.best_params_)
```

Output:

```
...  LINEAR REGRESSION RESULTS
     MSE: 24.291119474973513
     RMSE: 4.928602182665336
     R2 Score: 0.668759493535632
     Best Alpha (Ridge): {'alpha': 1}

     LOGISTIC REGRESSION RESULTS
     Accuracy: 0.8725490196078431
     Confusion Matrix:
      [[50  8]
      [ 5 39]]
     Classification Report:
                   precision    recall  f1-score   support

                0       0.91      0.86      0.88        58
                1       0.83      0.89      0.86        44

            accuracy                           0.87       102
           macro avg       0.87      0.87      0.87       102
        weighted avg       0.87      0.87      0.87       102

     Best Parameters (Logistic): {'C': 1}
```





Conclusion

Both Linear and Logistic Regression were successfully implemented using a single real-world dataset. The experiment demonstrated regression and classification tasks effectively, and hyperparameter tuning improved model performance.